第4章 自动化测试

《软件测试与质量保证》

马樱

Tel: 18359703871

y.ma@hit.edu.cn

计算学部 软件学院

测试面临的问题

- 测试用例越来越多，工作量越来越大，且许多测试用例会被不断地重复执行。如果由手工来完成，不仅占用很多人力资源，而且工作重复单调，会影响测试人员的积极性，降低测试工作人员的热情… 怎么办?

4.1 自动化测试的内涵

4.2 自动化测试的原理

4.3 测试工具

4.4 自动化测试的引入

什么是自动化测试

- 自动化测试（automated test）是相对手工测试（manual test）而存在的一个概念，由手工逐个地运行测试用例的操作过程被测试工具自动执行的过程所代替。
- 测试工具的使用是自动化测试的主要特征。

- 提问：根据统计结果，70%~85%的故障是由手工测试发现的，而自动化测试的产出较小，那么是不是可以不用自动化测试呢?

手工测试的局限性

- 无法确保能否覆盖所有代码和路径。
- 很难捕捉到与时序、死锁、资源冲突、多线程等有关的错误。
- 难以实施系统的负载/性能测试（因为需要模拟大量数据或大量并发用户）。
- 难以实施系统的可靠性测试（因为需要模拟系统运行十年、甚至几十年）。
- 难以在短时间内完成大量（如上千个）的测试用例（尤其是回归测试时）。
- 面对测试条件组合的爆炸数量，无能为力或效率非常低。

自动化测试的特点

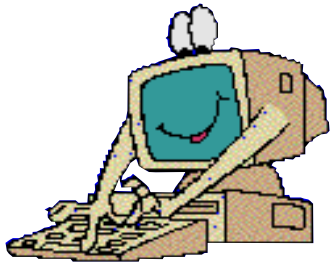
- 自动运行速度快，是手工测试无法相比的。
- 测试结果准确。比如测试搜索引擎网站时，即使搜索用时是微秒级的，测试工具也能发现差异。
- 高复用性。一旦形成测试脚本，则可运行多遍。
- 永不疲劳： 7天*24小时！
- 可靠： 机器不会故意撒谎，且手工测试比较容易出错。
- 独特的能力： 模拟运行环境（并发用户）。

自动化测试带来的好处

- 测试周期缩短
- 更高质量的产品
- 软件过程更规范
- 高昂的团队士气：测试人员有机会学习编程，获取新技术，测试工作更有趣，更有挑战性
- 节省人力资源，降低企业成本
- 充分利用硬件资源，降低企业成本

手工测试 VS 自动化测试

- 发现缺陷率高
- 容易实施
- 创造性、灵活性
- 覆盖率量化困难
- 重复测试效率低
- 不一致性、可靠性低
- 依赖人力资源



- ✓ 高效率（速度）
- ✓ 高复用性
- ✓ 覆盖率容易度量
- ✓ 准确、可靠
- ✓ 不知疲劳
- ✓ 激励团队士气
- ✓ 机械、难以发现缺陷
- ✓ 一次性投入大

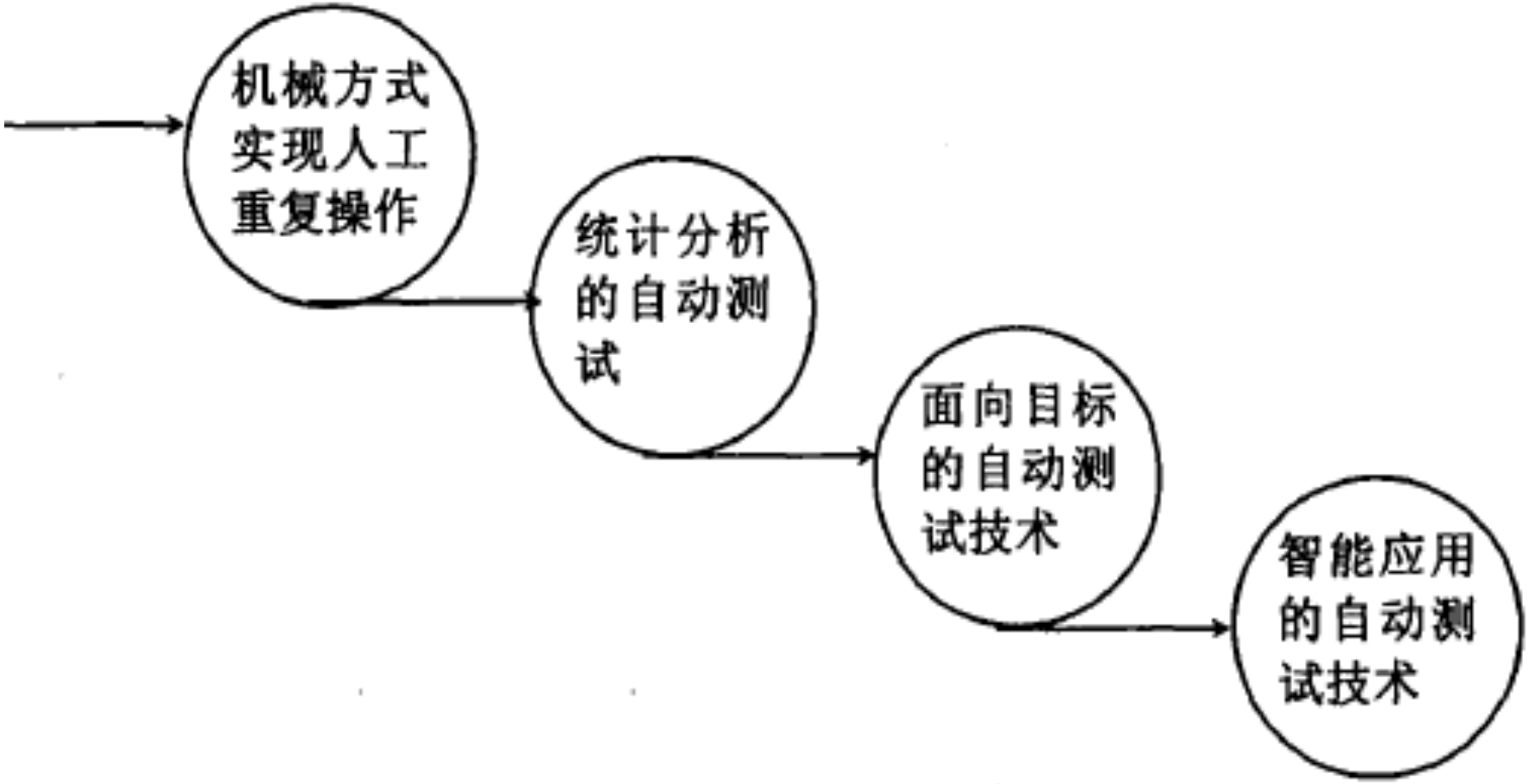
自动化测试在不同测试阶段的应用

技术	描述	自动化测试的应用特点
单元测试	保证软件质量的重要基础性工作，通常由开发人员完成。典型的单元测试方法是测试驱动开发：在编写代码之前先完成测试用例，所有测试通过的时候，开发也就完成了。	在单元测试中，自动化测试和手工测试交叉进行，代码扫描（静态测试）可以是全自动的，而动态测试需要手工干预和调试，属于半自动的。
冒烟测试	冒烟测试（**Building Verification Test**）是为了保证后续的大量测试可以正常进行而对基本功能进行的验证测试。	在良好的软件构建环境中，如每日集成、每日构建等，**BVT**通常可以实现完全自动化。
功能测试	包括基本操作、数据输入和输出的验证、逻辑合理性的推敲、用户界面友好性识别等。	基本操作和数据验证等可以实现自动化测试，但适用性测试一般由手工完成。
性能测试	对系统模拟不同的负载测试，从而发现系统的性能瓶颈、或确定具体的性能指标。	一般只能由自动化测试工具完成。
回归测试	重复已经存在的测试，对原有正常工作的验证，以避免其他地方的代码改动对原有功能的负面影响。	自动化测试最能发挥作用的地方，应尽量实现自动化。
验收测试	通过执行用户场景或模拟真实用户场景，使用系统以证明系统满足用户所期望的功能而进行的测试。一般用户也参与验收测试。	自动化测试可以进行，但宜采用手工测试。

两者互为补充

- 工具本身缺乏想象力和创造性，自动化测试只能发现15%的缺陷，需要手工测试！
  - 在系统功能逻辑测试、验收测试、适用性测试，以及涉及交互性测试时，多采用手工测试方法；
  - 单元测试、集成测试、系统负载或性能、可靠性测试等比较适合采用自动化测试；
  - 对那种不稳定、开发周期短或一次性的软件等则不适合自动化测试。

自动化测试的发展历程

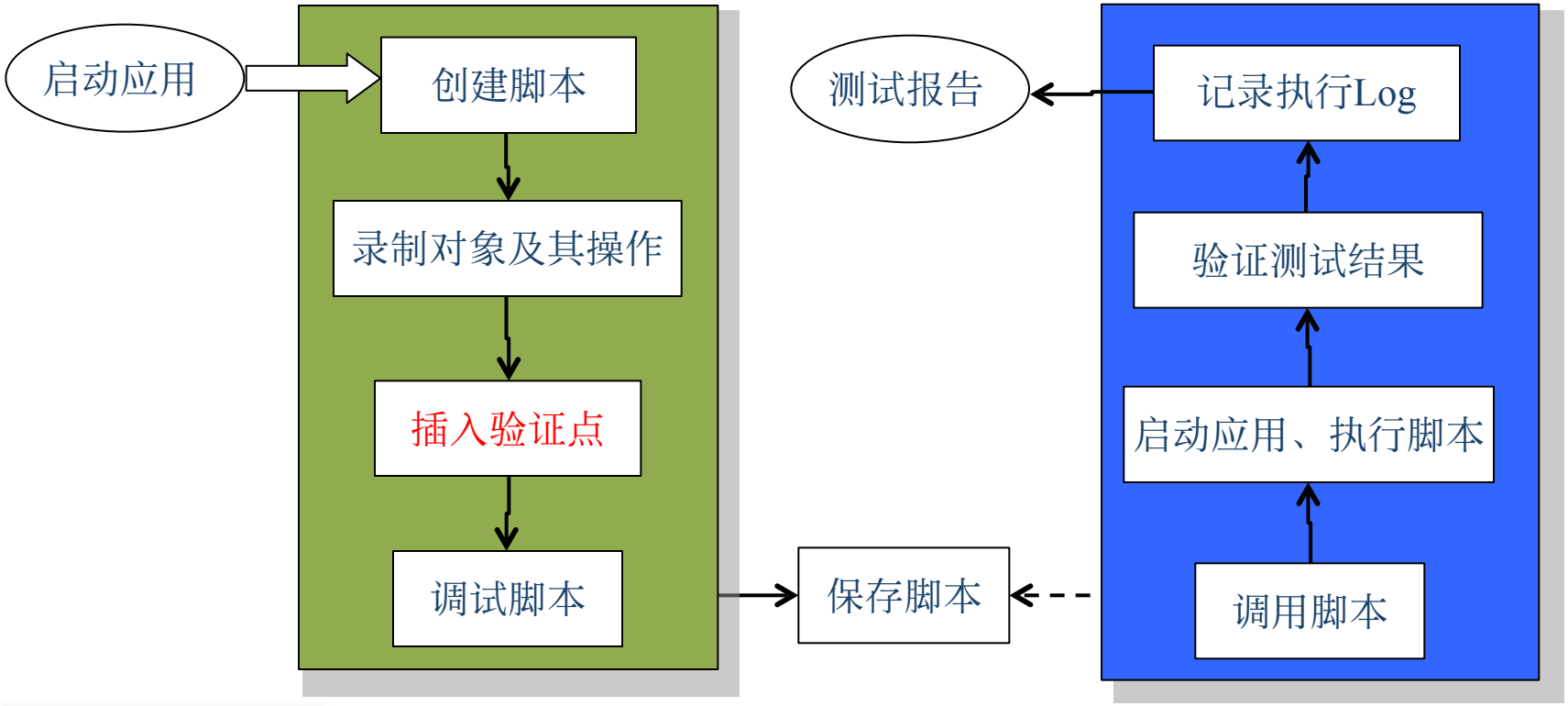


自动化测试的类型

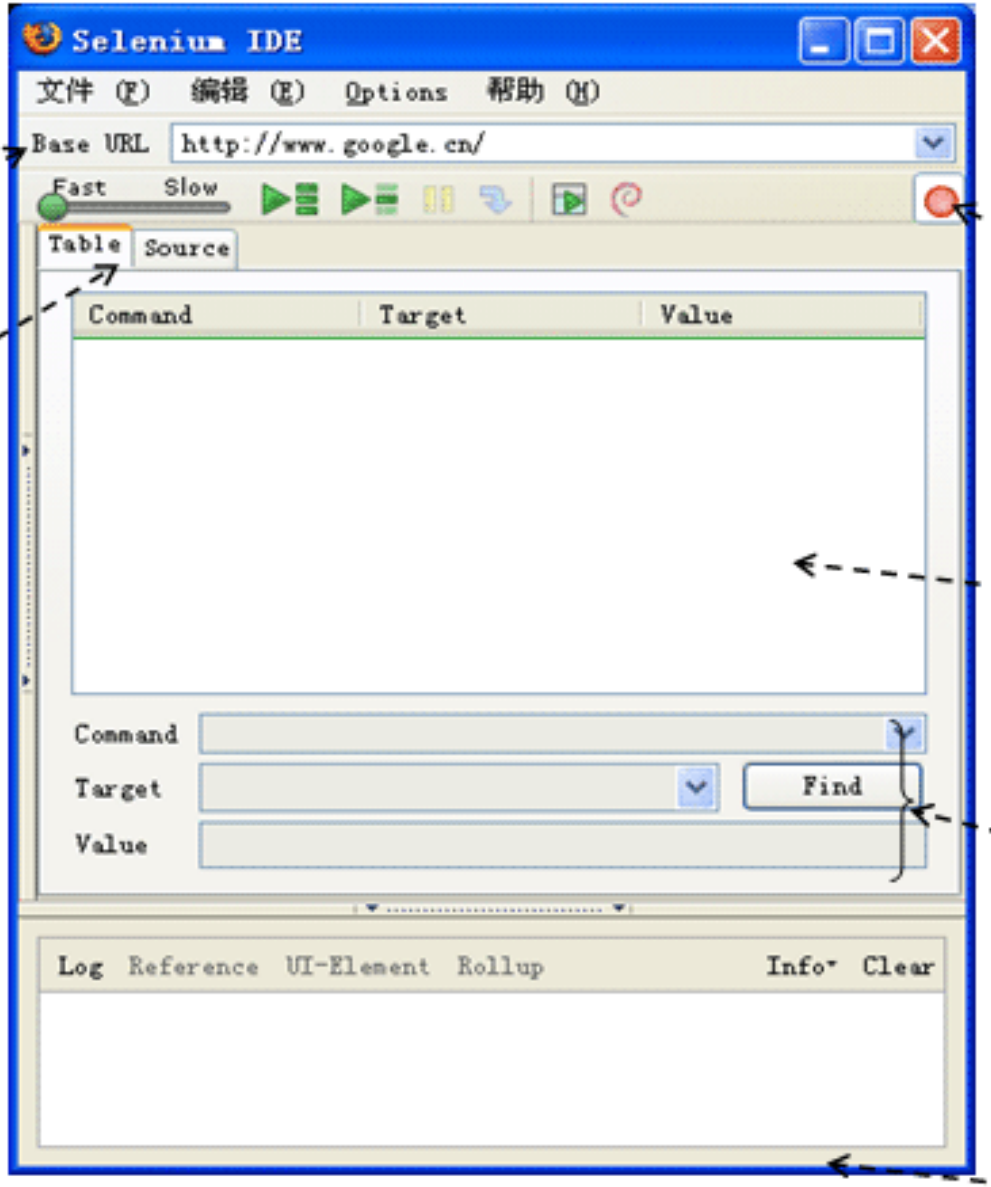
- 代码扫描（静态测试，白盒方式）
- 录制/回放（常用于GUI测试，黑盒方式）
- 监控类（常用于可靠性/安全性测试）：
  - 设置断点，检测变量的值
  - 内存泄漏
  - 网络情况
- 模拟类（常用于负载/压力测试）：
  - 并发
  - 大数量

基于脚本的自动化测试流程

测试脚本（**test script**）是测试工具执行的一组指令集合，使计算机能自动完成测试用例的执行，也是计算机程序的一种形式。



一个自动化测试的例子



基准URL

脚本显示的两种方式

操作按钮

脚本窗口

命令参考和编辑窗口

运行日志窗口

软件工程系

Selenium IDE *

文件 (F) 编辑 (E) Options 帮助 (H)

Base URL http://www.google.cn/

Fast Slow

Table Source

Command	Target	Value
assertTitle	Google	
type	q	用Selenium进行自动化测试
clickAndWait	btnG	
assertTitle	用Selenium进行自动化测试 - Googl…	
verifyTextPresent	www.ibm.com	
verifyTextPresent	9,310项符合	
verifyTextPresent	搜索用时 0.34 秒	
click	link=用Selenium 自动化验收测试	

Command type

Target q Find

Value 用Selenium进行自动化测试

Log Reference UI-Element Rollup Info Clear

[info] Executing: |assertTitle | Google | |

[info] Executing: |type | q | 用Selenium进行自动化测试 |

[info] Executing: |clickAndWait | btnG | |

[info] Executing: |assertTitle | 用Selenium进行自动化测试 - Google 搜索 | |

[info] Executing: |verifyTextPresent | www.ibm.com | |

[info] Executing: |verifyTextPresent | 9,310项符合 | |

[info] Executing: |verifyTextPresent | 搜索用时 0.34 秒 | |

[error] false

[info] Executing: |click | link=用Selenium 自动化验收测试 | |

[warn] Link has target '_blank', which is not supported in Selenium! Randomizing target to be: selenium_blank49163

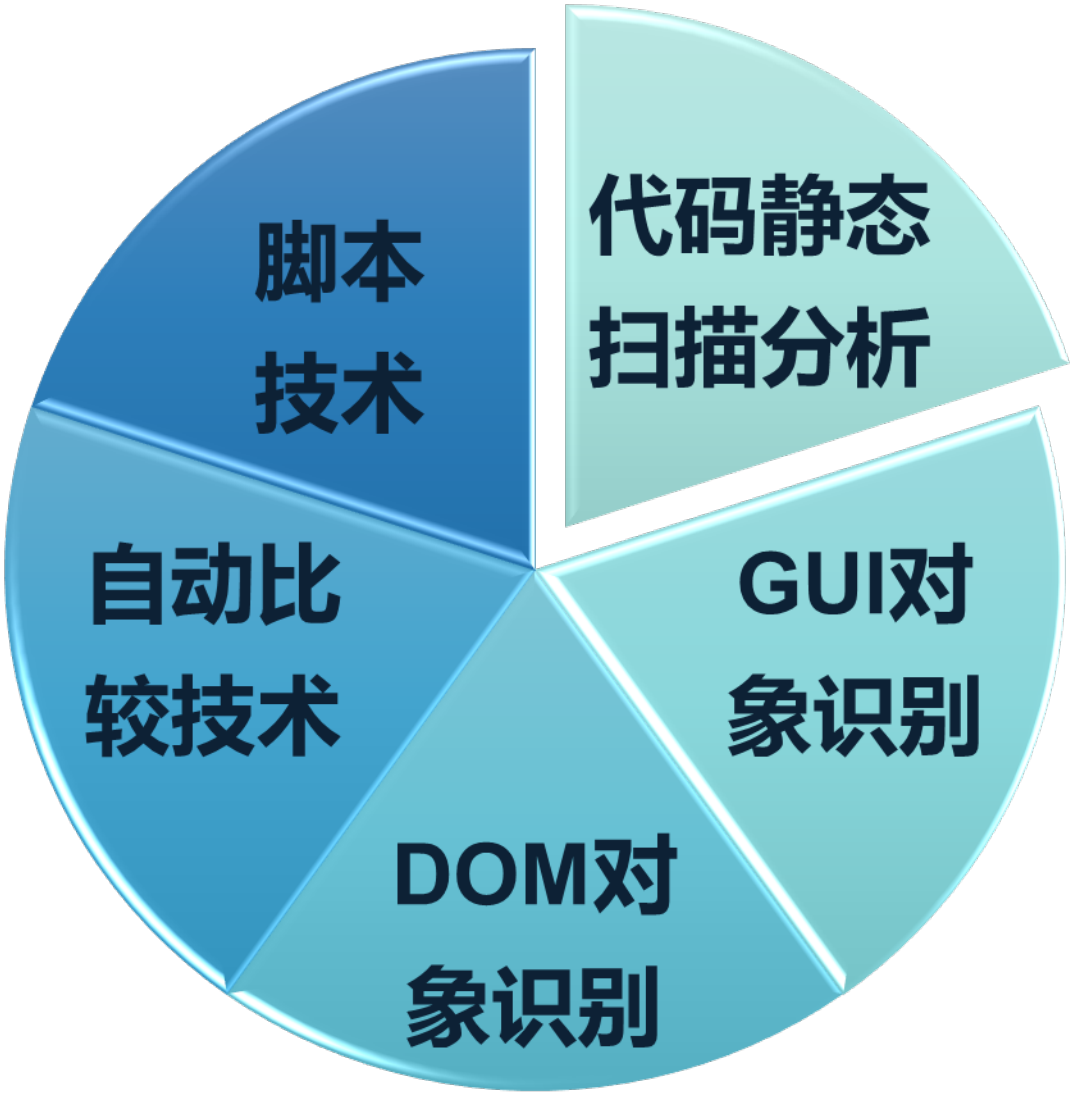
4.1 自动化测试的内涵

4.2 自动化测试的实现原理

4.3 测试工具

4.4 自动化测试的引入

涉及的多种实现技术



1、代码的静态分析/扫描

- 代码静态分析是在不运行程序的情况下，仅通过对源代码进行扫描和分析（如其语法、结构、过程、接口等），便可找出其中隐藏的问题，如：
  - 参数不匹配
  - 错误的递归
  - 非法计算
  - 可能出现的空指针引用
  - 多余的变量
  - ……
- 思考：静态分析是不是在干编译器的工作?

代码的静态分析/扫描(续)

- 事实上，静态分析可做的工作远大于编译器。
- 利用代码分析，可以验证源代码是否满足规范性、安全性、可靠性、可维护性等指标。

规则！

❖ 代码分析的步骤：

1. 定义规则
2. 扫描代码，抽取特征参数
3. 比较特征参数是否匹配规则

❖ 代码分析的关键是建立各种规则，规则的来源：

- 编程语言的语法
- 编程规范
- 编程的最佳实践（经验）……

规则的例子

- 例如，针对Java语言，可依据扩展巴科斯-诺尔范式(EBNF)的语法来建立规则。

addition: INTEGER ‘+’ INTEGER;

加法是一个整数后面跟着一个‘+’字符和另一个整数。

INTEGER: (‘0’..’9’)+;

整数是由0至9之间的数字（至少出现一次）所组成的字符。

- 许多支持代码静态分析的测试工具会提供预定义好的规则，例如Parasoft Jtest提供了700多个。

Parasoft Jtest的规则

Help – Parasoft Jtest

Search: GO Search scope: All topics

Contents

- Jtest Static Analysis
- BugDetective (License Required) [BD]
  - Exceptions [BD.EXCEPT]
    - Avoid NullPointerException [BD.EXC
    - Avoid ClassCastException [BD.EXCEP
  - Resource Leaks [BD.LEAKS]
  - Optimization [BD.OPT]
  - Possible Bugs [BD.PB]
  - Security [BD.SECURITY]
- JavaBeans [BEAN]
- Code Duplication Detection [CDD]
  - Avoid code duplication [CDD.DUPC]
  - Avoid method duplication [CDD.DUPM]
  - Avoid string literal duplication [CDD
  - Avoid class duplication [CDD.DUPT]
  - Avoid duplicate import statements [CD
- Coding Conventions [CODSTA]
- Design by Contract [DBC]
- Eclipse Development [ECLIPSE]
- Enterprise JavaBeans [EJB]
- Exceptions [EXCEPT]
- Formatting [FORMAT]
- Garbage Collection [GC]
- Global Static Analysis [GLOBAL]
- Hibernate Framework [HIBERNATE]
- Initialization [INIT]
- Internationalization [INTER]
- Java 2 Micro Edition [J2ME]
- Javadoc Comments [JAVADOC]
- Java Database Connectivity [JDBC]
- JavaServer Page Metrics [JMETRICS]
- JavaServer Pages [JSP]
- JUnit Test Case [JUNIT]
- Class Metrics [METRICS]

Jtest Static Analysis

Avoid string literal duplication [CDD.DUPS-2]

DESCRIPTION

This rule identifies code with duplicated string literals. The rule will find duplicated string literals across all tested code, whether or not the duplicated literals appear in the same file. An error is reported for each occurrence.

NOTES

N/A

SECURITY RELEVANCE

N/A

PARAMETERS

- Minimum number of occurrences to be considered a violation (default value is 5)
- Ignore length of string less than (default value is 2)

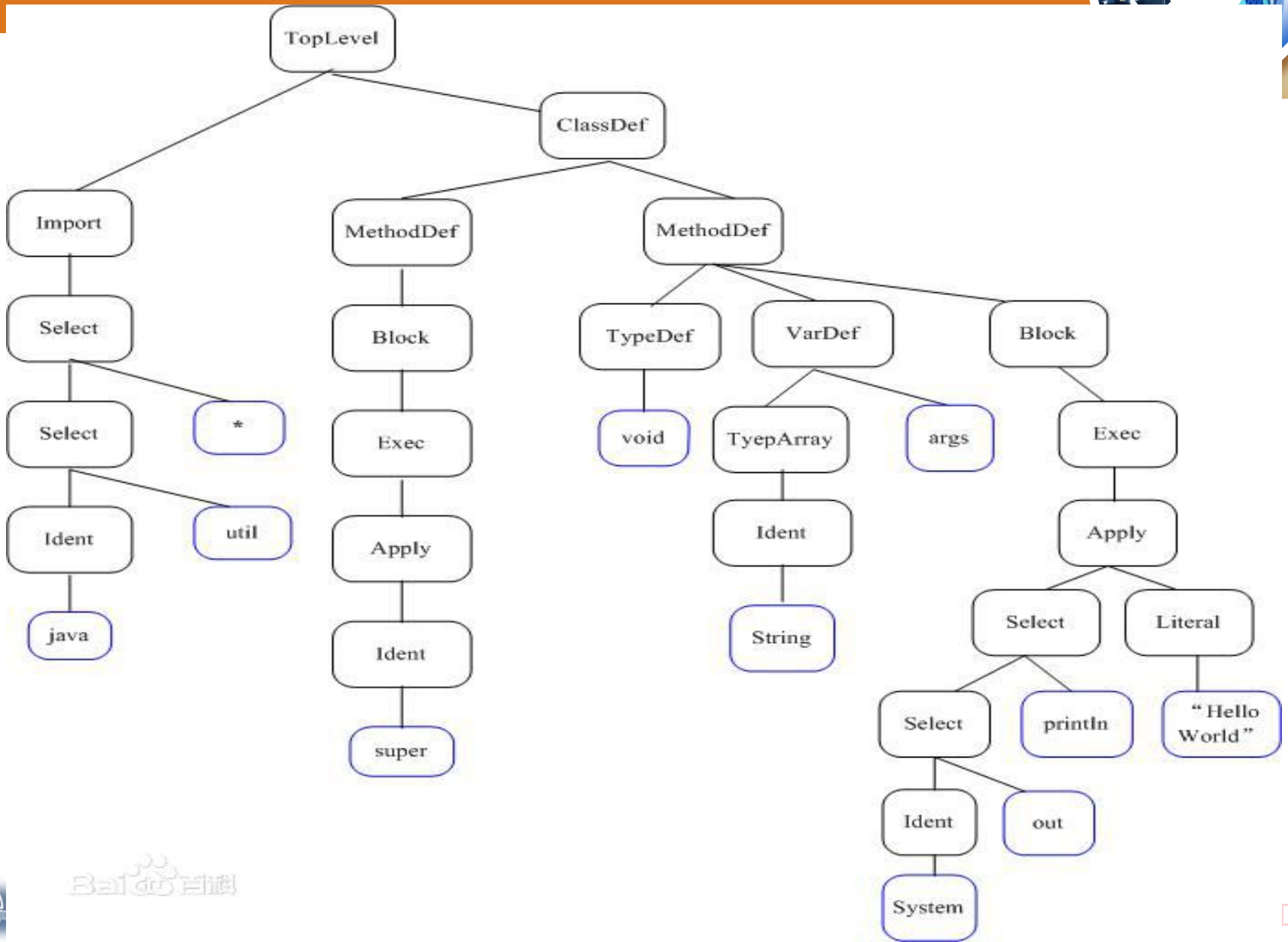
BENEFITS

If a string literal is used repeatedly, it should be stored in a constant. It would be much easier for a developer to change the value of a repeated string by changing one single string constant instead of finding all the repeated string liter and changing them one by one.

代码分析的技术

1. 词法分析：用正则表达式抽取标识符，如**Lex**。
2. 语法分析：使用上下文无关语法将标识符整理成抽象语法树**AST**，如**Yacc**。
3. 语义分析：对结构上正确的源程序进行上下文有关性质的审查（如类型审查）。
4. 控制流分析：分析状态图或函数调用关系图，其孤立的节点部分为无效代码。
5. 数据流分析：对数据流图进行遍历，记录变量的初始化点和引用点，保存相关数据信息。
6. 污点分析：基于数据流图判断源代码中哪些变量可能受到攻击，是验证程序输入、识别代码表达缺陷的关键。
7. 缺陷模式匹配：内置一些共性的缺陷模式。
8. 类型推断：对代码中运算对象类型进行推理，从而保证代码中每条语句都针对正确的类型执行。

一棵抽象语法树


TopLevel
ClassDef
Import
MethodDef
MethodDef
Select
Block
TypeDef
VarDef
Block
Select
*
Exec
void
TyepArray
args
Exec
Ident
util
Apply
Ident
Apply
java
Ident
String
Select
Literal
Select
println
"Hello World"
super
Ident
out
System

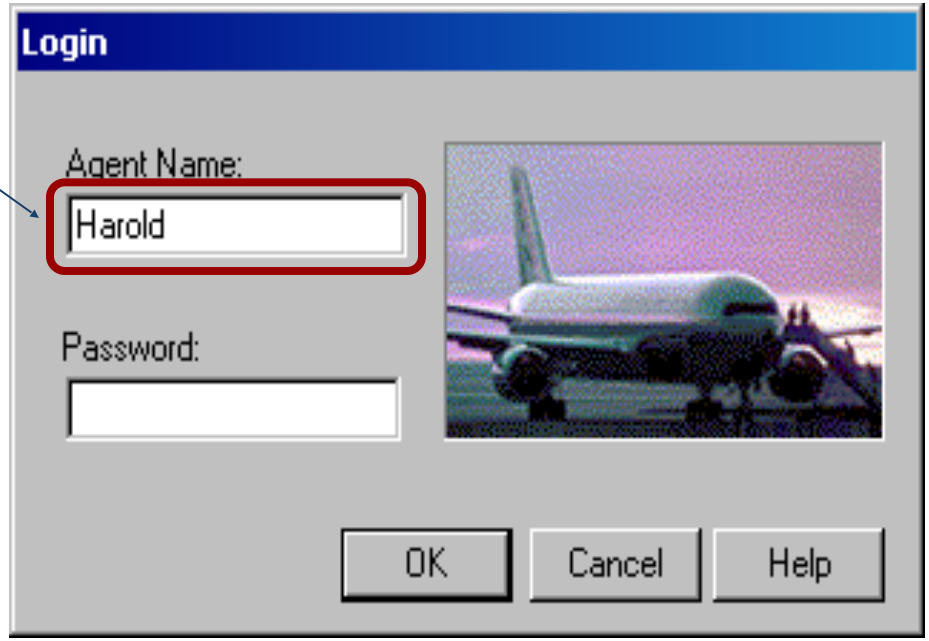
静态扫描能发现的问题

- 错用了常量、局部变量和全局变量；
- 调用了并不存在的子程序；
- (接口)函数调用出现了不匹配的参数或返回值；
- 不适当的处理顺序、递归、嵌套或死循环；
- 未定义的变量或定义了但未使用的变量；
- 输入参数没有进行合法性检查；
- 缺少适当的出错处理代码；
- 永远不会执行到的代码；
- 不适当的连接；
- 算法还可继续优化；
- 违背了编码规则、命名规则或编码风格。

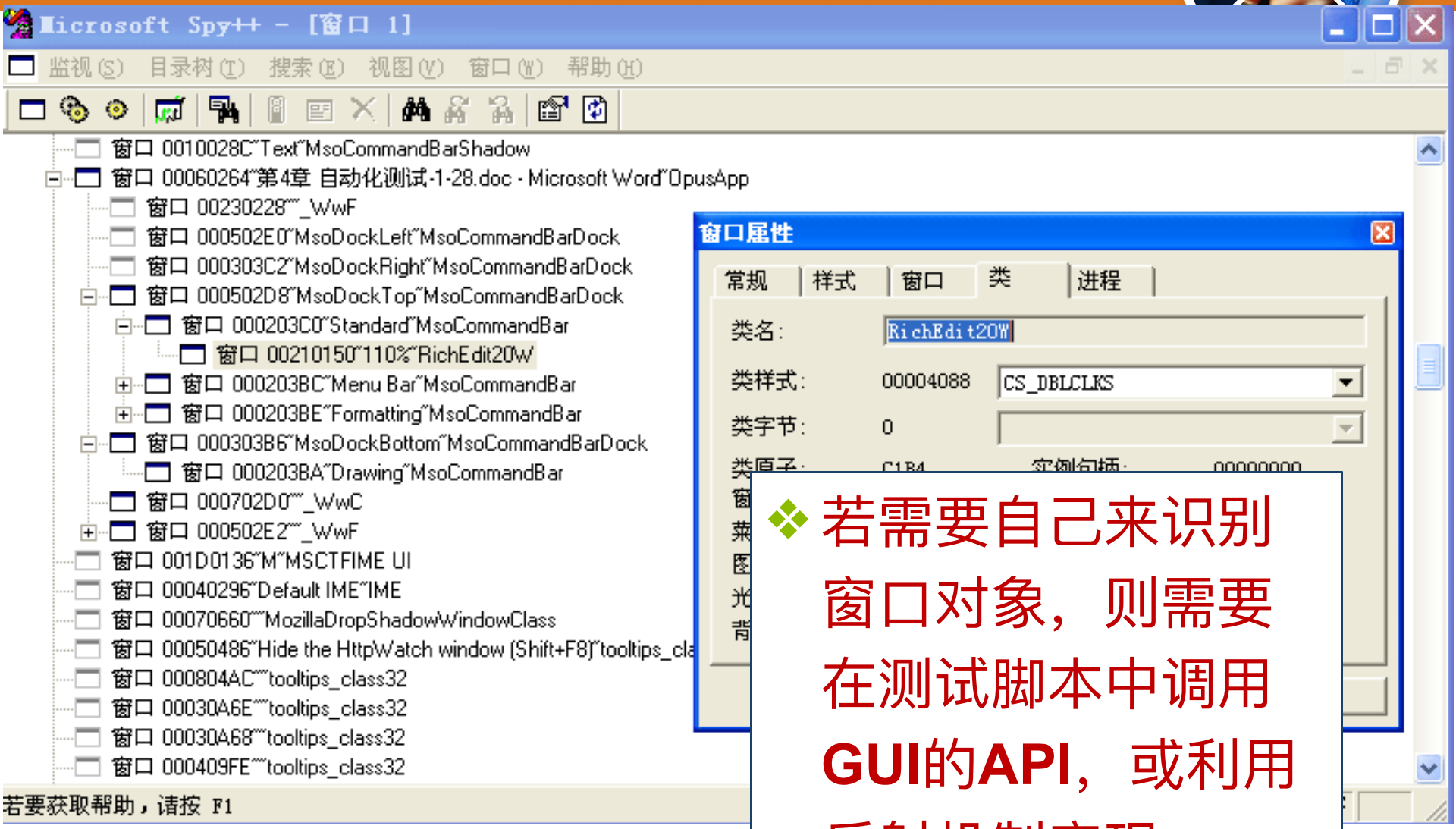
2、GUI对象的识别

- 功能测试时，往往需要自动化工具能识别和控制用户界面上的各种对象。
- 对象识别就是获得对象的类型、名称、属性值。

Nativeclass:	Edit
Attached Text:	Agent Name
Enabled:	True
Focused:	True
Height:	20
Width:	119
Text:	Harold

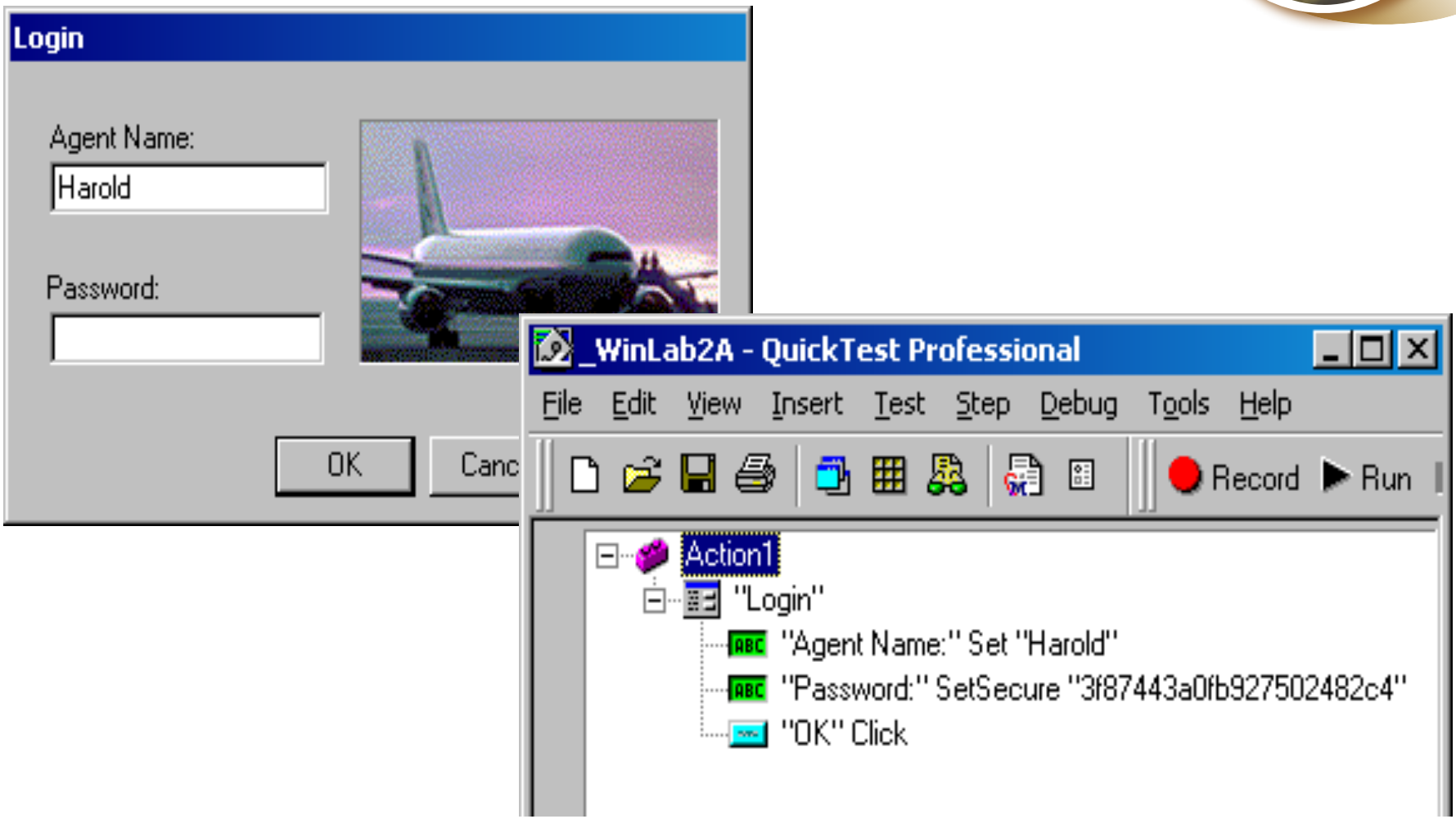


使用MS Spy++识别窗口中的对象

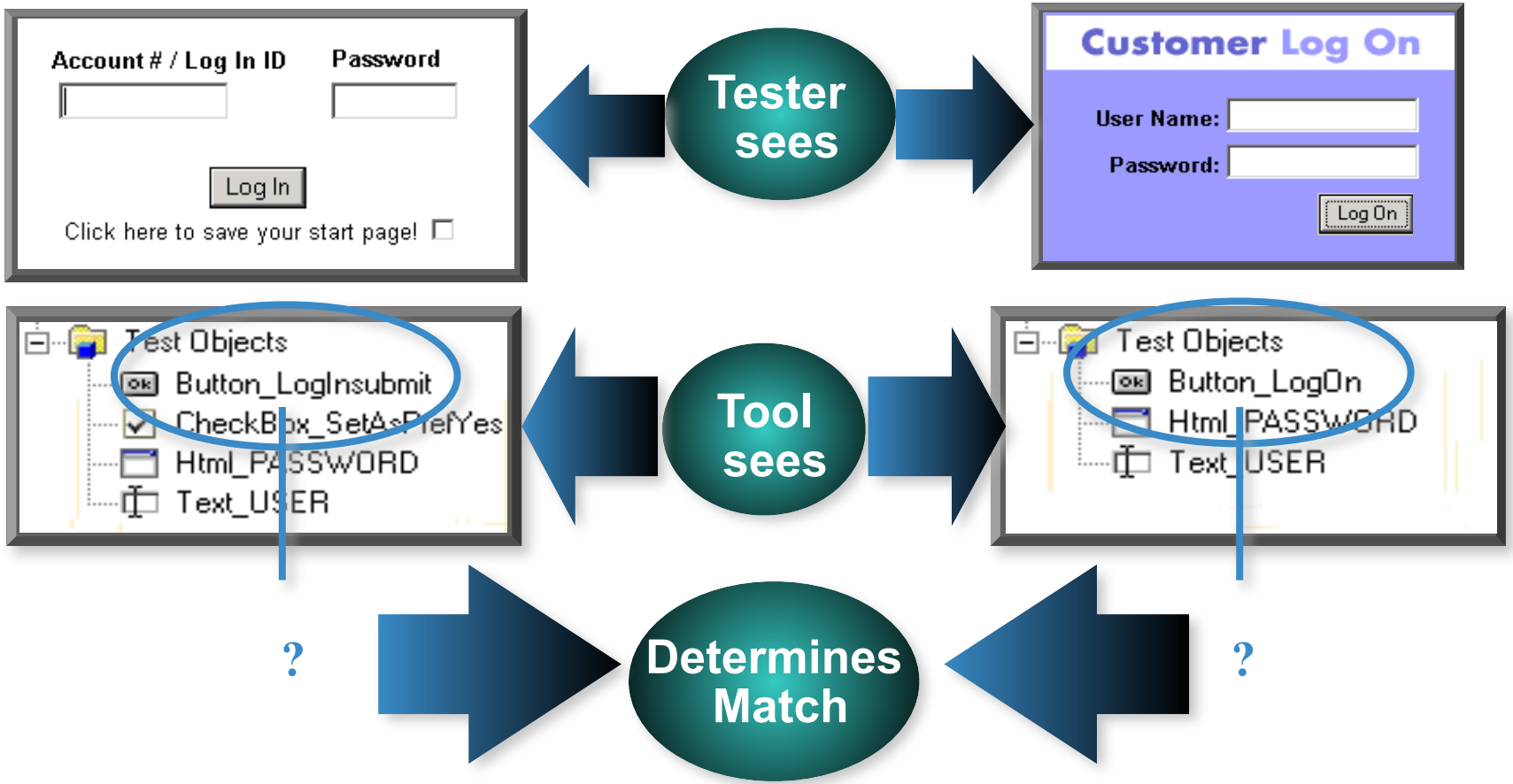


- 若需要自己来识别窗口对象，则需要在测试脚本中调用**GUI的API**，或利用反射机制实现。

例： GUI对象的操作


Login
Agent Name:
Harold
Password:
OK
Canc
_WinLab2A - QuickTest Professional
File Edit View Insert Test Step Debug Tools Help
Record
Run
Action1
"Login"
"Agent Name:" Set "Harold"
"Password:" SetSecure "3f87443a0fb927502482c4"
"OK" Click

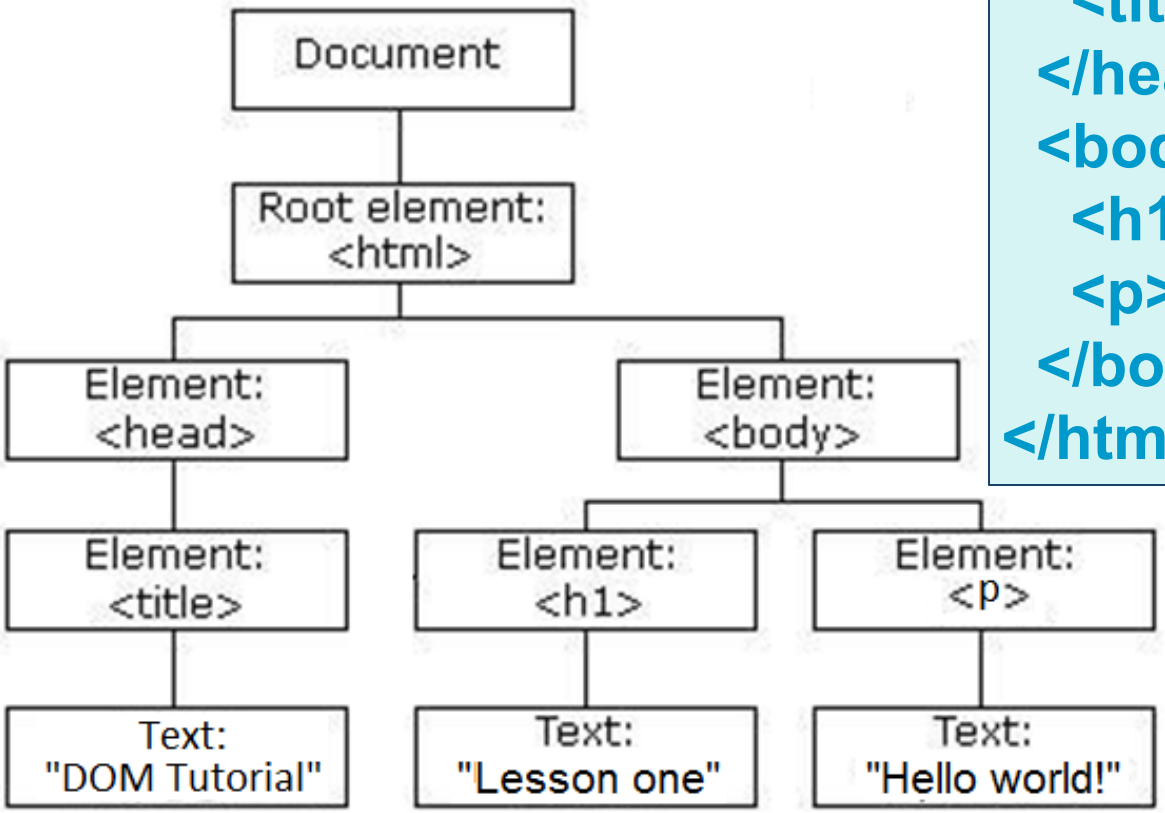
对象映射的问题


Account # / Log In ID
Password
Log In
Click here to save your start page!
Tester sees
Customer Log On
User Name:
Password:
Log On
Test Objects
Button_LogInsubmit
CheckBox_SetAsPrefYes
Html_PASSWORD
Text_USER
Tool sees
Test Objects
Button_LogOn
Html_PASSWORD
Text_USER
?
Determines Match
?

3、DOM对象的识别

- 对于web页面，需要识别其中的DOM对象。
- DOM：即Document Object Model，文档对象模型。
- DOM是W3C推荐的处理超文本标记语言(HTML)和可扩展标志语言(XML)的标准编程接口。
  - 核心DOM：针对任何结构化文档的标准模型
  - XML DOM：针对XML文档的标准模型
  - HTML DOM：针对HTML文档的标准模型
- DOM提供了一系列的API，可访问HTML/XML文件中各对象的属性、方法和事件。

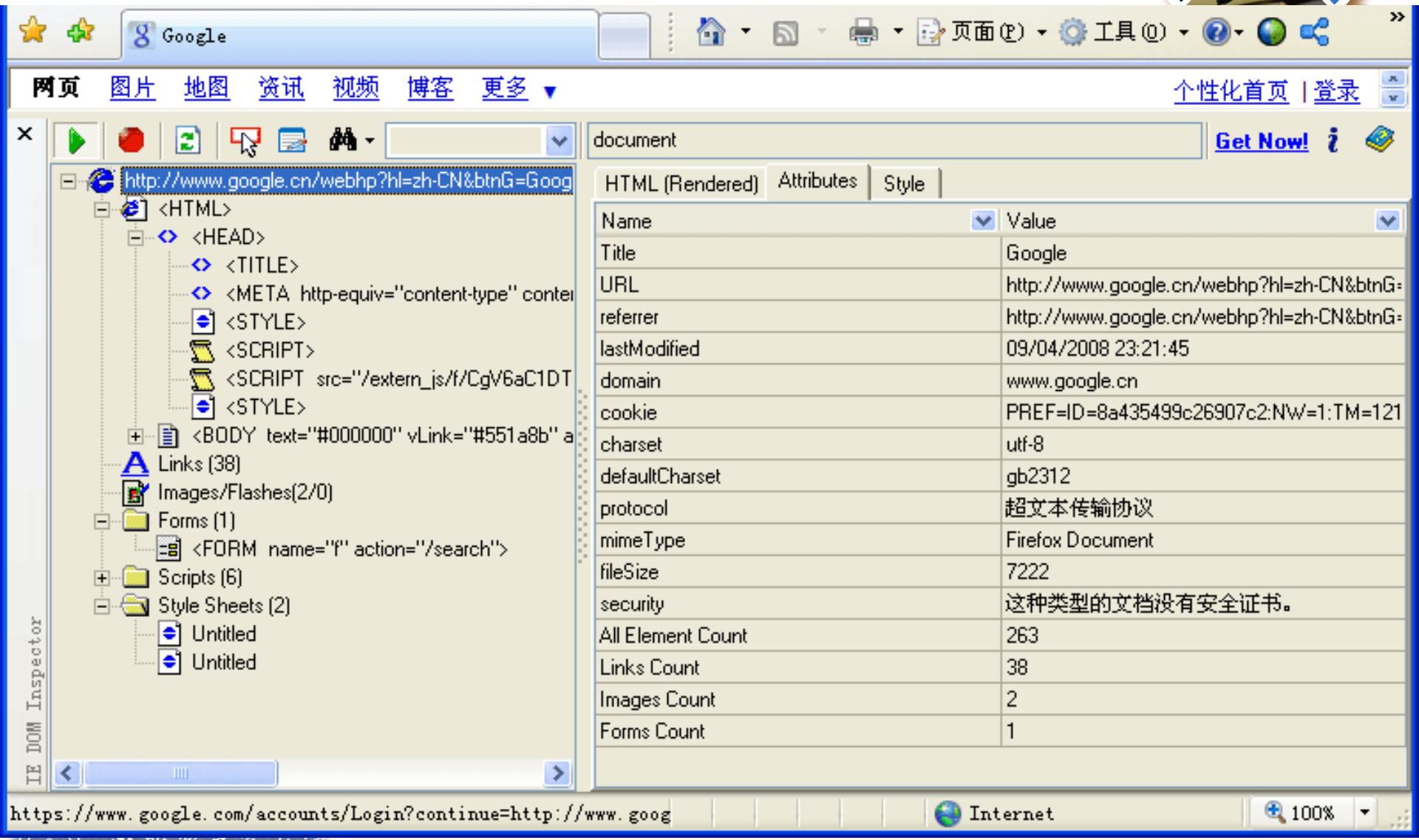
DOM节点树




```
<html>
  <head>
    <title>DOM Tutorial</title>
  </head>
  <body>
    <h1>Lesson one</h1>
    <p>Hello world!</p>
  </body>
</html>
```

例： **document.getElementsByTagName("p");**

示例：IE DOM Inspector



示例：FireBug

查看 | window

控制台 HTML CSS 脚本 DOM 网络 Test

⊞ google	Object kEI=ermASfWLHIausALc6_3iDA kEXPI=17259,18167
⊞ XPCSafeJSObjectWrapper	XPCSafeJSObjectWrapper()
⊞ _gjp	_gjp()
⊞ _gjuc	_gjuc()
⊞ onload	onload(event)
⊞ postMessage	postMessage()
⊞ rwt	function()
⊞ sf	function()
⊞ showModalDialog	showModalDialog()
⊞ Components	nsXPCComponents interfaces=nsXPCComponents_Interfaces
⊟ document	Document webhp?rls=ig
readyState	"complete"
⊞ activeElement	activeElement()
⊞ elementFromPoint	elementFromPoint()
⊞ enableStyleSheetsForSet	enableStyleSheetsForSet()
⊞ getElementsByClassName	getElementsByClassName()
⊞ hasFocus	hasFocus()
⊞ lastStyleSheetSet	lastStyleSheetSet()
⊞ preferredStyleSheetSet	preferredStyleSheetSet()
⊞ selectedStyleSheetSet	selectedStyleSheetSet()

4、自动比较

- 测试时必须将实际结果与期望结果进行比较，自动化测试时也应实现自动化的结果比较。
- 需要比较的内容和形式：
  - 数字、文本
  - 屏幕区域的图像
  - 窗口或窗口中控件的数据或属性值
  - 网页
  - 文件
- 通常，期望结果都是预先定义好的，保存在测试脚本、数据库或外部文件中。

比较技术

- 直接比较：精确的比较
  - 通常是针对简单的变量类型
  - 如：数值、字符串是否完全相同?
- 智能比较：模糊的比较，允许有差异
  - 如：判断输出报表中是否包含日期，即只关心日期的格式，但不关心到底是哪天。
  - 可采用正则表达式、屏蔽等搜索技术。
- 由于比较的正确率和效率会直接影响到测试的成败，通常在比较之前最好先用比较过滤器先进行数据的预处理。

数据的动态匹配

- 在某些情况下，输出数据可能是动态的：
  - 可以利用模式匹配技术来验证数据
  - 如：验证**Order ID ###**，而不是**Order ID 230**
  - 此法可考虑到各种不同的合法响应

Selenium中验证命令的示例

验证点命令	对象	值
verifyTitle	My Page	
verifyValue	nameField	John Smith
verifySelected	dorpdown2	value=js*123
verifyTextPresent	You are now logged in	
verifyAttribute	txt1@class	bigAndBlod
verfyVisible	postcode	

Robot验证点

- ComboBoxVP
- EditBoxVP
- FileVP
- HTMLLinkVP
- HTMLDocumentVP
- ListViewVP
- WindowVP
- JavaListViewVP

例：**WindowVP方法**

Result* = WindowVP *(action%, recMethod$, parameters$)

其中，***action*** 可以是：

CompareImage

CompareMenu

CompareProperties

Exists/DoesNotExist…

如：

WindowVP (CompareImage, "Caption=Paint", "VP=PICT1A")

此方法执行通过则返回**1**，执行失败则返回**0**。

5、脚本技术

- 测试脚本(**test script**)是测试工具执行的一组指令集合，使计算机能自动完成测试用例的执行，也是计算机程序的一种形式。
- 可以通过录制产生，也可以直接用脚本语言编写。
- 可用来编写测试脚本的脚本语言
  - **python**
  - **vbs**
  - **TCL**
  - **Perl**
  - **ruby……**

- 测试脚本的类型：
  - 线性脚本
  - 结构化脚本
  - 数据驱动脚本
  - 关键字驱动脚本

（1）线性脚本

- 线性脚本是最简单的一种脚本，只能顺序执行。
- 录制测试步骤所产生的脚本都是线性脚本，它将记录下用户所做的键盘和鼠标操作。
  - 直接录制形成的脚本可以完整的回放执行，适合在回归测试中使用。
  - 若测试步骤一致，但输入数据有不同，则还需要修改脚本。
  - 也可在线性脚本中增加一些简单的指令，如增加等待时间、作比较等，但都不会改变顺序执行的性质。

```
Sub Main
    Dim Result As Integer

    Window SetContext, "Class=Shell_TrayWnd", ""
    PushButton Click, "Text=开始"

    Window SetContext, "Caption=「开始」菜单", ""
    ListView Click, "ObjectIndex=2;\;ItemText=控制面板(C)", "Coords=57,17"

    Window SetContext, "Caption=控制面板", ""
    ListView DblClick, "Text=FolderView;\;ItemText=显示", "Coords=37,30"

    Window SetContext, "Caption=显示 属性", ""
    TabControl Click, "ObjectIndex=1;\;ItemText=桌面", ""
    PushButton Click, "ObjectIndex=2"

    Window SetContext, "Class=#32770", ""
    GenericObject Click, "Class=Static;ClassIndex=1", "Coords=77,58"

    Window SetContext, "Caption=显示 属性", ""
    TabControl Click, "ObjectIndex=1;\;ItemText=屏幕保护程序", ""
    EditBox Click, "Label=等待(W):", "Coords=23,9"
    EditBox Left_Drag, "Label=等待(W):", "Coords=22,9,31,9"
    InputKeys "6"
    ComboBox Click, "ObjectIndex=1", "Coords=160,11"
    ComboListBox Click, "ObjectIndex=1", "Text=贝塞尔曲线"
    PushButton Click, "Text=设置(T)"

    Window SetContext, "Caption=贝塞尔曲线屏幕保护程序设置", ""
    SpinControl Click, "ObjectIndex=1", "Coords=8,5"
    SpinControl DblClick, "ObjectIndex=2", "Coords=9,7"
    ScrollBar HScrollTo, "ObjectIndex=1", "Position=13"
    PushButton Click, "Text=确定"

End Sub
```

例：线性脚本

（2）结构化脚本

- 线性脚本适用范围窄，相似的脚本数量多，单个脚本的重用性不高，实际应用得较少。
- 结构化脚本类似于结构化程序：
  - 包含顺序、选择分支、循环等逻辑结构
  - 支持调用子脚本
- 结构化脚本还可以分层、分模块的形式组织，甚至可以将底层的脚本发展成基础的库脚本。

例：结构化脚本

```
;Include 常量
#include <GUIConstants.au3>

;初始化全局变量
Global $GUIWidth
Global $GUIHeight

$GUIWidth = 300
$GUIHeight = 250

;创建窗口
GUICreate("New GUI", $GUIWidth, $GUIHeight)
......
While 1
  ;检查用户点击窗口中哪个按钮
   $msg = GUIGetMsg()

   Select
     Case $msg = $GUI_EVENT_CLOSE
          GUIDelete()
          Exit
     Case $msg = $OK_Btn
          MsgBox(64, "New GUI", "You clicked on the OK button!")
     Case $msg = $Cancel_Btn
          MsgBox(64, "New GUI", "You clicked on the Cancel button!")

   EndSelect

WEnd
```

软件工程系

（3）数据驱动脚本

- 对于线性脚本和结构化脚本，其测试输入和比较是“捆绑”在脚本中的，重用性不高。
- 数据驱动脚本将测试执行步骤和测试输入数据进行了分离：
  - 测试脚本中只有测试执行相关的指令；
  - 测试的输入数据存储在独立的数据文件中。
- 同一个数据驱动脚本可以针对不同的输入数据进行测试，提高了脚本的使用效率和可维护性。

测试数据列表（Datatable）

序号	用户名	口令
1	Test	Pass1
2	test sp	pass1
3	test	pass 1
4	test	P@ss!

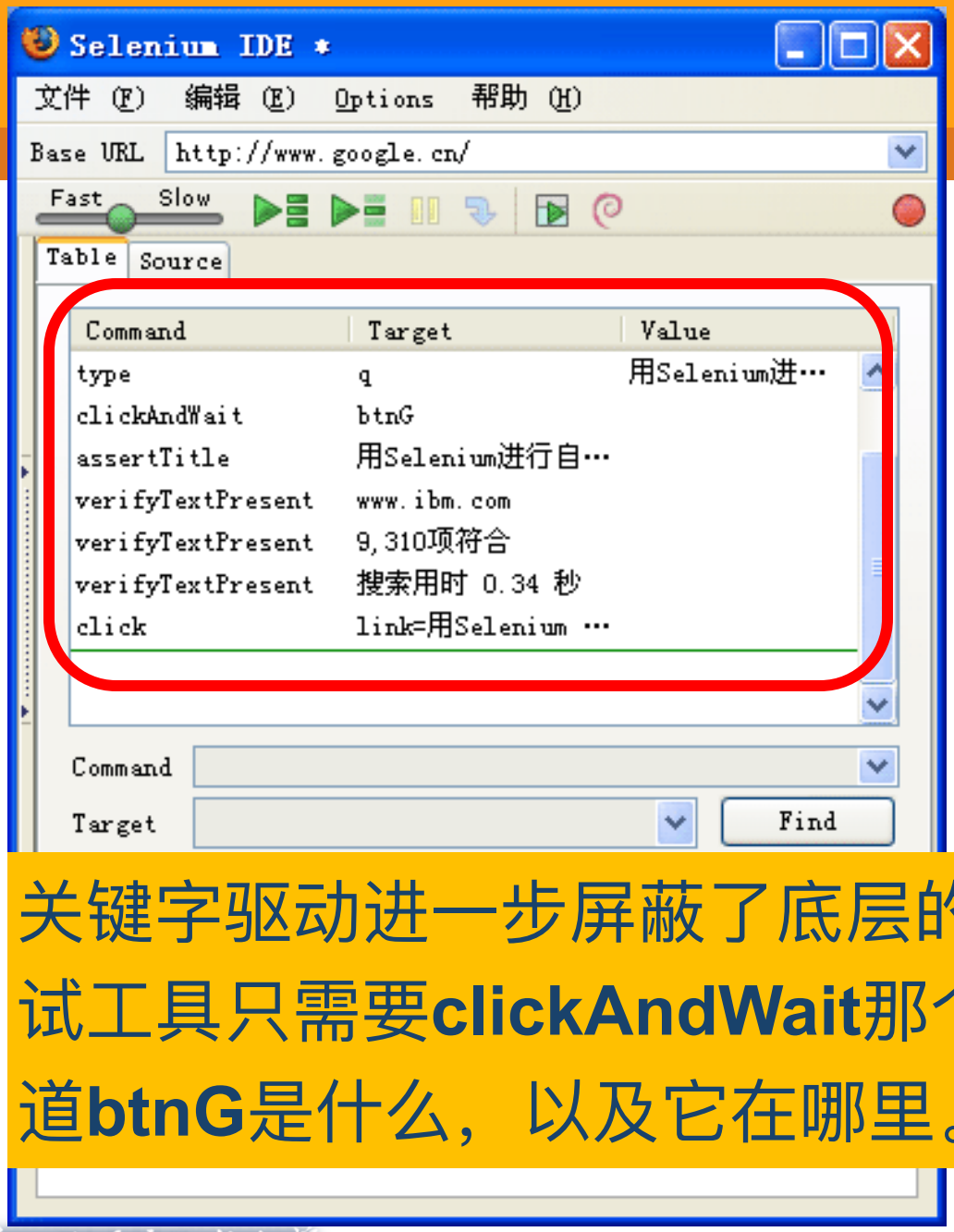
例：数据驱动脚本

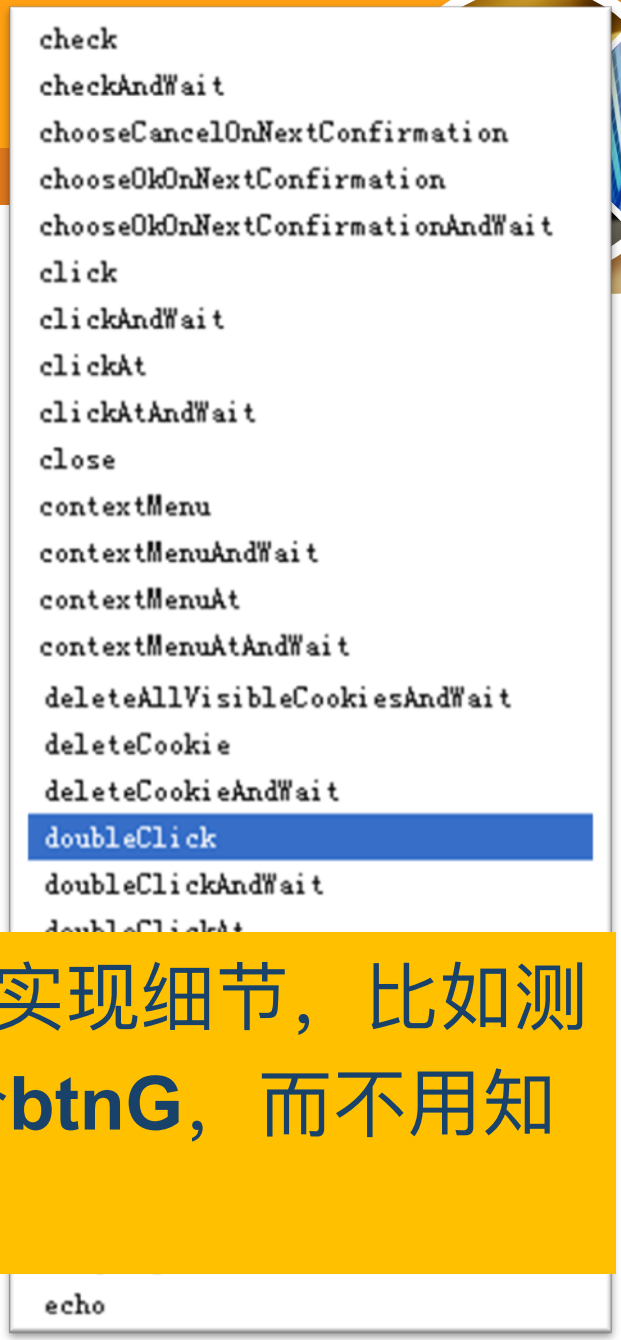
```
For i=1 to Datatable.GetRowCount
    Dialog("Login").WinEdit("AgentName:").SetDataTable("username", dtGlobalSheet)
    Dialog ("Login") .WinEdit ("Password:") .SetDataTable ("passwd", dtGlobalSheet)
    Dialog ("Login") .WinButton ("OK") .Click
    datatable.GlobalSheet.SetNextRow
Next
```

（4）关键字驱动脚本

- 数据驱动脚本仅将输入数据从测试逻辑(步骤)中分离出来，关键字驱动脚本则更进了一步——将测试逻辑分解成“操作对象”和“关键字”，形成表格形式的脚本。

窗口	对象	行为	参数
LoginPage	UserIDField	InputText	“admin”
LoginPage	PasswordField	InputText	“123”
LoginPage	SubmitButton	Click	





关键字驱动进一步屏蔽了底层的实现细节，比如测试工具只需要**clickAndWait**那个**btnG**，而不用知道**btnG**是什么，以及它在哪里。

关键字驱动脚本（续）

❖ 关键字驱动脚本的编写步骤：

1. 识别用户界面上的控件，形成对象库；
2. 分析各对象需要进行哪些操作、输入什么样的测试数据——即测试用例；
3. 填写表格，形成自动化测试脚本。

❖ 使用关键字驱动脚本的好处：

- 只要确定了用户界面上的控件，即便业务流程的代码还没实现，也可以开始编写测试脚本。

——测试先行！

关键字！

- 关键字驱动的脚本使用一系列关键字描述了测试和验证的步骤。
- 实际上，每个关键字都封装了相应的函数，但测试人员不需要了解这些基础的函数。
- 也可以将结构化的逻辑引入，比如：

:FOR	${var}	IN	@{SOME LIST}
	Run Keyword If	'${var}' == 'EXIT'	Exit For Loop
	Do Something	${var}	

4.1 自动化测试的内涵

4.2 自动化测试的实现原理

4.3 测试工具

4.4 自动化测试的引入

测试工具的分类

- 根据测试方法不同，分为：
  - 白盒测试工具和黑盒测试工具
  - 静态测试工具和动态测试工具
- 根据测试的对象和目的，分为：
  - 单元测试工具、功能测试工具、性能测试工具
  - 测试管理工具
- 根据工具的来源不同，分为：
  - 开源测试工具和商业测试工具
  - 自主开发的测试工具和第三方测试工具

常见的测试工具

	HP Mercury	IBM Rational	Micro Focus Borland	Microsoft	开源软件	其它
(C++, Java) 单元测试		PurifyPlus	DevPartner	VSTS (Visual Studio Team System)	xUnit (JUnit, CppUnit) CppTest JWebUnit Jasmine	QA•C++ QA•J C++Test JTest
静态分析	Code Advisor	Software Architect Application Developer	DevPartner	CodeRush NDepend StyleCop	FindBugs CheckStyle PMD CppLint	JSLint Cppdepend Clang
功能测试	Unified Functional Testing (QTP)	Functional Tester	Silk Test	VSTS	Selenium Jameleon	SoapUI QARun TestComplete
性能测试	LoadRunner; StormRunner Load	Performance Tester	Silk Performer; WebMeter	VSTS	JMeter Grinder Gatling	WebLoad QALoad

常见的测试工具（续）

	HP Mercury	IBM Rational	Micro Focus Borland	Microsoft	开源软件	其它
安全性测试工具	Fortify	AppScan	DevPartner	Security Essentials	Metasploit WebScarab	Knocwork Coverity
移动测试	AppPulse Mobile	Test Workbench	Silk Mobile	VSTS; Hopper Test Tool	Robotium Espresso Calabash Appium UIAutomator	Ranorex; Experitest SeeTest
敏捷测试	Sprinter	Team Concert	Borland Agile	VSTS	Cucumber FitNesse	ThoughtWorks Twist
覆盖率分析工具	Code Coverage Tool	Code Coverage	DevPartner	NCover	Cobertura EclEmma CovTool	Atlassian Clover; BullseyeCoverage Testwell CTC++
缺陷管理	Test Director	ClearQuest	Start Team	VSTS	MantisBT BugZilla	JIRA Fogbugz
测试管理	ALM /QC	Quality Manager; Test Realtime	Silk Central	VSTS	TestLink Testopia	TestLodge QMetry qTest

HP Mercury的主要测试工具

- **LoadRunner**：性能测试工具，可模拟上千万的并发用户数，从而实施性能的测试和预测。
- **WinRunner**：适用于企业级(大型)应用的功能测试工具，提供录制、检测和回放等功能。
- **QTP(QuickTest Professional)**：以关键字驱动技术实施的自动测试工具。
- **TestDirector**：基于web的测试管理工具，包含了需求管理、测试计划、故障库等。

Loadrunner

HP LoadRunner Controller - Scenario1 - [Design(1)]

File View Scenario Results Diagnostics Tools Help

Scenario Groups

	Group Name	Script Path	Quantity	Load Generators
☑	test	E:\lingbao\script\test	10	localhost

Service Level Agreement

New Details Edit Delete

Service Level Agreement

Currently no SLA rules are defined for the load test.
Click the New button to define SLA criteria for your load test.

Scenario Schedule

Schedule Name: Schedule 1

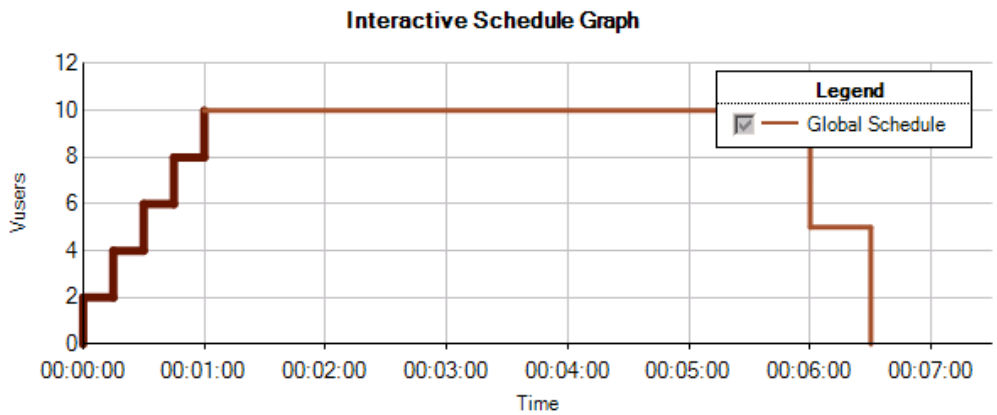
Schedule by: Scenario Group

Run Mode: Real-world schedule Basic schedule

Global Schedule

Total: 10 Vusers

Action	Properties
Initialize	Initialize each Vuser just before it runs
Start Vusers	Start 10 Vusers: 2 every 00:00:15 (HH:MM:SS)
Duration	Run for 00:05:00 (HH:MM:SS)
Stop Vusers	Stop all Vusers: 5 every 00:00:30 (HH:MM:SS)



Design Run

Auto Collate Resu

IBM Rational的主要测试工具

- **Robot**：功能/性能测试工具，支持三种脚本，即用于功能测试的**GUI**脚本、用于性能测试的**VU**脚本和**VB**脚本。
- **PurifyPlus**：内存监测工具，支持**C**、**VC++**、**VB**、**Java**代码。
- **ClearQuest**：故障库。
- **PureCoverage**：覆盖率分析工具。
- **Testmanager**：测试管理工具。

123 - Rational Robot - [example]

File Edit View Record Debug Insert Tools Window Help

Verification Points
- Object Data
- Object Data2
- Object Propert
- Object Propert

```
'$Include "CheckDBHeader.sbh"

Sub Main
    Dim Result As Integer

    'Initially Recorded: 2007-6-18  14:06:01
    'Script Name: example
    StartApplication """"D:\program files\Rat:

    Window SetContext, "Name=frmExistingLogi
    PushButton Click, "Name=cmdOK"

    Window SetContext, "Name=frmMain", ""
```

Build | Console

00001 | 001 | admin

Compuware的主要测试工具

- QARun：功能测试工具。
- QALoad：性能测试工具。
- TrackRecord：故障库。
- QACenter：测试管理工具。

其它的一些商业性工具

- 单元测试：
  - Parasoft公司的Jtest和C++Test
  - PR公司的QA•C和QA•C++
- 功能测试：
  - Segue公司的SilkTest
  - Oracle公司的Empirix e-Test Suite
  - Parasoft公司的WebKing
- 性能测试：
  - Borland公司的SilkPerformer
  - Radview公司的WebLoad

开源的一些测试工具

- 静态分析：Splint、FindBugs
- 单元测试：JUnit
- 功能测试：
  - Selenium、AutoIT、AutoHotkey
  - Abbot Java GUI Test Framework
- 性能测试：Jmeter
- 网络监控：Wireshark/Ethereal、Netcat、Snort
- 故障库：Mantis
- 测试管理：TestLink

http://www.opensourcetesting.org

FindBugs

- FindBugs是由美国马里兰大学提供的一款开源 的Java静态代码分析工具。
- FindBugs以插件形式插入Eclipse，通过检查类文件或JAR文件，将字节码与定义好的一组缺陷模式进行对比，从而发现代码缺陷，完成静态代码分析。

Java - Java project one/src/testSampleCalculator.java - Eclipse Platform

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explore | Hierarchy | JUnit

Finished after 0.25 seconds

Runs: 2/2 Errors: 0 Failures: 1

testSampleCalculator [Runner: JUnit 4] (0.062 s)
- testAdd (0.047 s)
- testSubtration (0.000 s)

Failure Trace

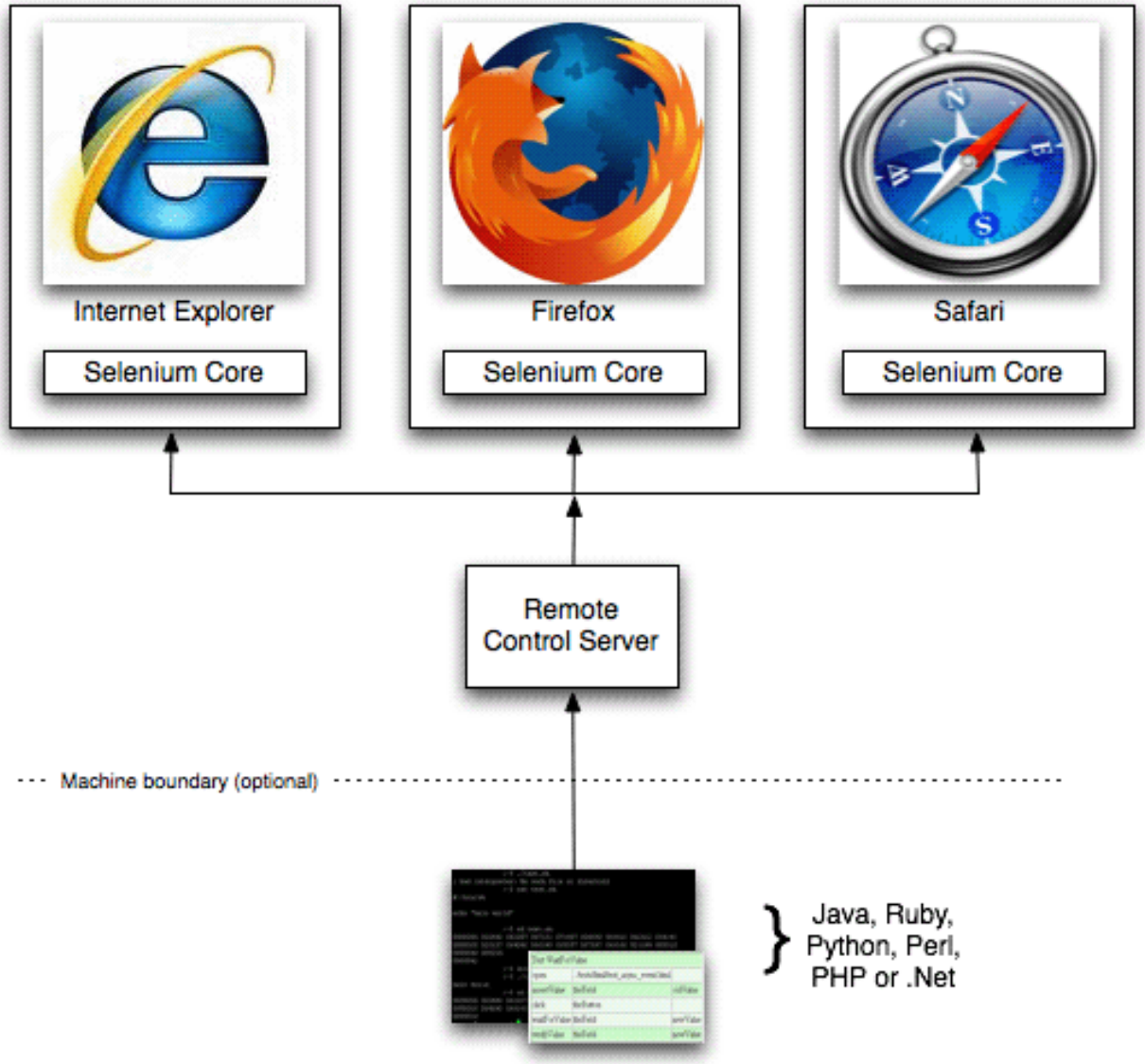
junit.framework.AssertionFailedError: expected:<72>
at testSampleCalculator.testAdd(testSampleCalculato

JUnit

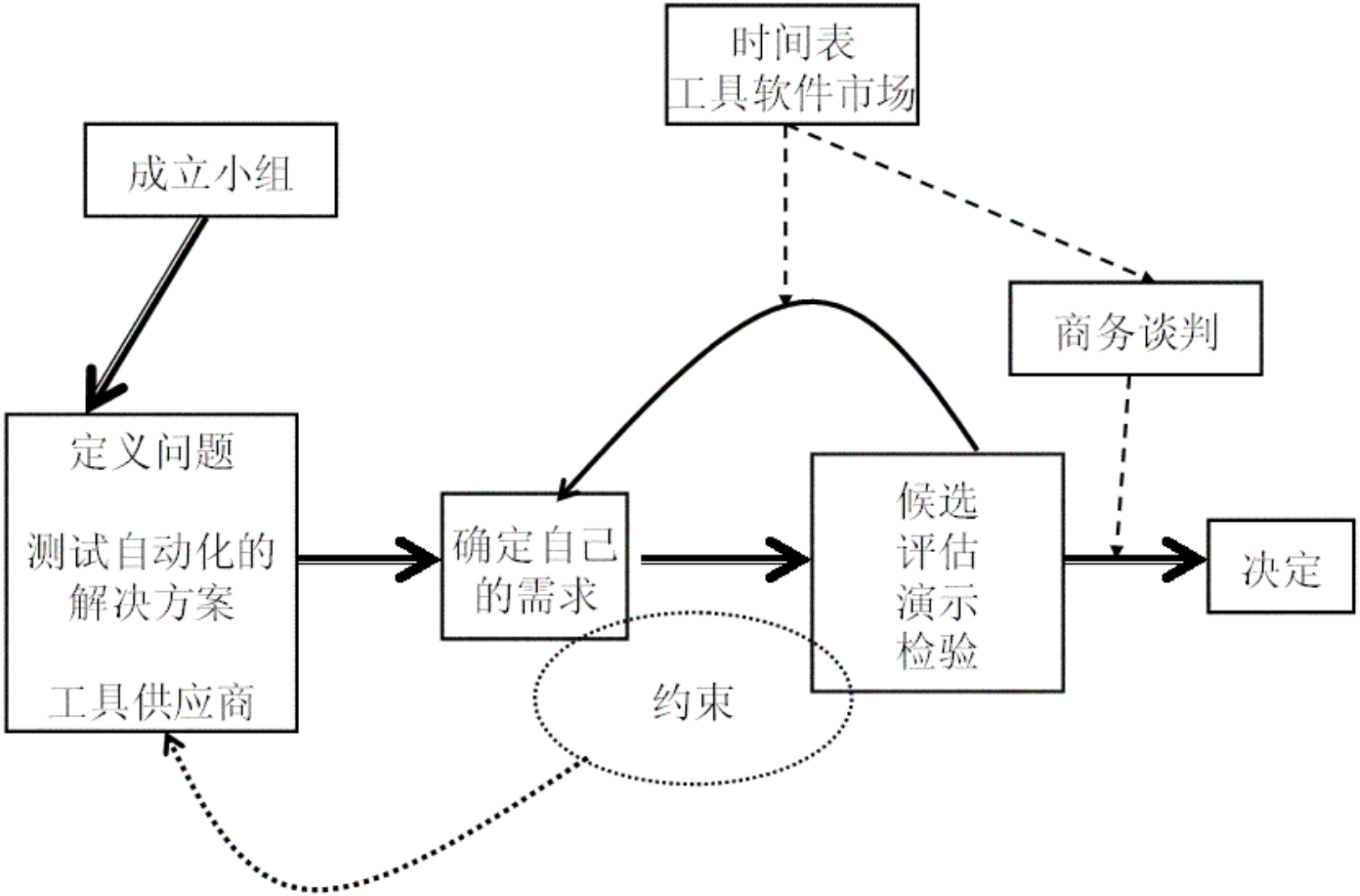
testSampleCalculator | SampleCalculator.jav

```
public class testSampleCalculator extends TestCase
{
    @Before
    public void setUp() throws Exception {
    }

    @After
    public void tearDown() throws Exception {
    }

      public void testAdd()
      {
            SampleCalculator calculator = new SampleCa
            int result = calculator.add(50 , 20);
             assertEquals(72 , result);
      }

      public void testSubtration()
      {
            SampleCalculator calculator = new Sample
            int result = calculator.subtration(50 , 
            assertEquals(30 , result);
      }

}
```

Selenium


Internet Explorer
Selenium Core
Firefox
Selenium Core
Safari
Selenium Core
Remote
Control Server
Machine boundary (optional)
Java, Ruby,
Python, Perl,
PHP or .Net

测试工具的选择


时间表
工具软件市场
成立小组
商务谈判
定义问题
测试自动化的
解决方案
工具供应商
确定自己
的需求
候选
评估
演示
检验
决定
约束

4.1 自动化测试的内涵

4.2 自动化测试的实现原理

4.3 测试工具

4.4 自动化测试的引入

正确认识自动化测试

- 不现实的期望注定自动化测试的失败
- 自动化测试能够：
  - 显著降低重复手工测试的时间
  - 建立可靠、重复的测试，减少人为错误
  - 增强测试质量和覆盖率
- 自动化测试不能够：
  - 完全替代手工测试和手工测试工程师
  - 保证100%的测试覆盖率
  - 弥补测试实践的不足

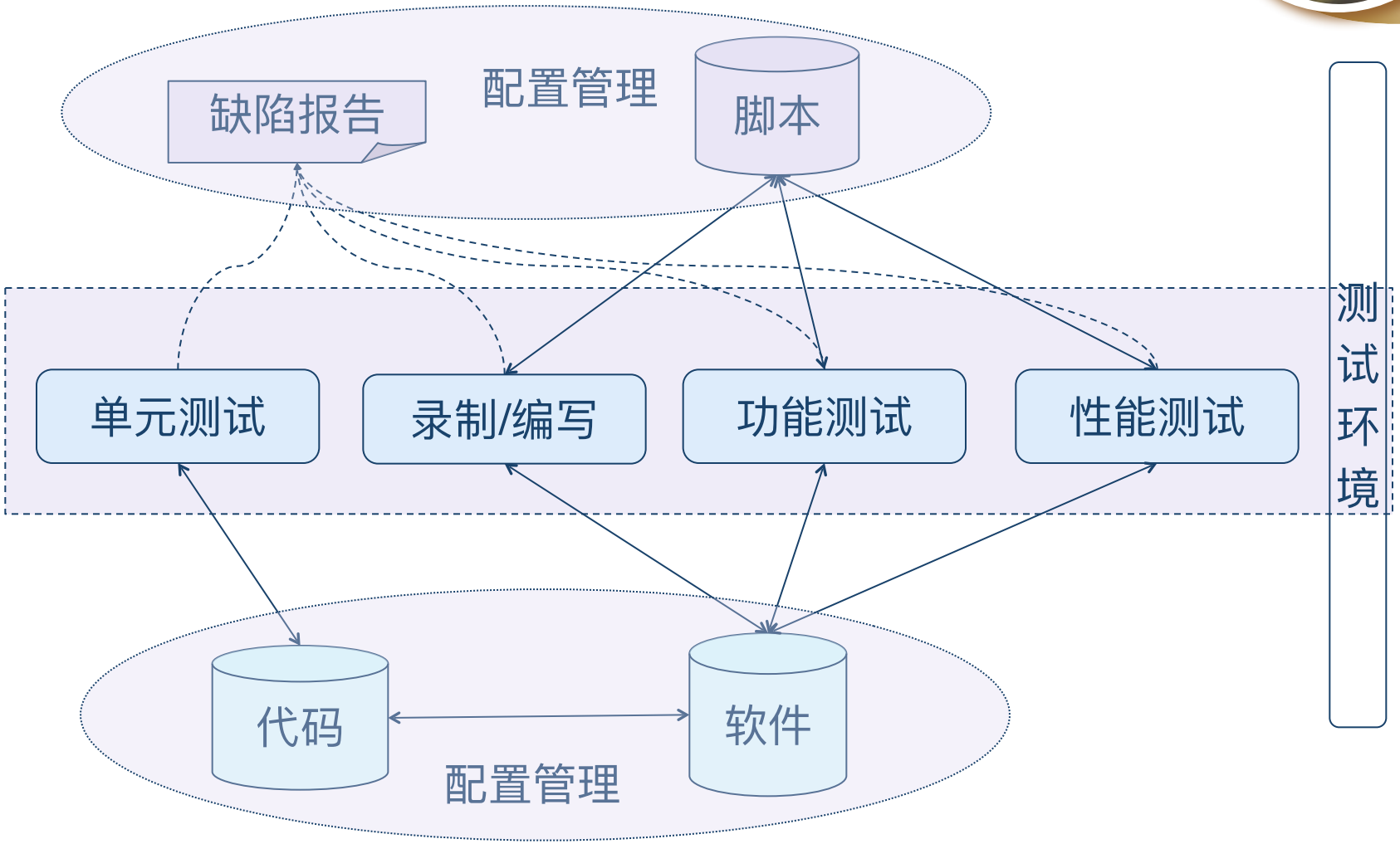
普遍存在的问题

- 不正确的观念或不现实的期望
- 缺乏具有良好素质、经验的测试人才
- 测试工具本身的问题影响测试的质量
- 测试脚本的质量低劣
- 没有进行有效的、充分的培训
- 没有考虑到公司的实际情况，盲目引入测试工具
- 没有形成一个良好的使用测试工具的环境
- 其它问题

对策

- 正确的认识
- 找准测试自动化的切入点
- 把测试脚本开发纳入整个软件开发体系
- 软件程序开发和测试自动化不可分离
- 资源的合理调度
- 测试自动化依赖测试流程和测试用例
- 降低测试自动化的投入、提高其产出

解决方案



IBM的解决方案

Rational Unified Process

TestManager

Robot/RobotJ

ClearQuest

测试计划 → 测试设计 → 测试开发 → 测试执行 → 测试评估

配置管理 - ClearQuest and ClearCase LT

Purify | Coverage | Quantify

Q & A

The End